

[Download the article](#)

Lesson 0 install root and software on your compute

If you get the THU's computer cluster account, use following commands to get into.

```
$ ssh -XY -p 48571 -o ServerAliveInterval=5 yourname@hepthu.com
```

You can also run code on your pc if you have installed root, it's up to you.

[homebrew](#) Click this link to go to the Homebrew page, follow the instructions to install Homebrew on your Mac.

Homebrew is a software package manager that allows you to download many applications on your Mac!

After finishing the installation, paste the following command in a macOS terminal to install CERN ROOT software: `$ brew install root`. This way, you can also install applications like `firefox` which you can't find in the App Store.

Additionally, `$` indicates a shell command, so drop the `$` sign when you copy the command.

Lesson 1 base commands for Linux

Some simple commands you can have a try!

```
1 pwd #print work directory
2 cd /path/to/directory #change directory
3 cd .. #back to parent directory
4 ls #list directory contents
5 mkdir test #make a directory
6 touch test.txt #creat a new txt file
7 cp test.txt copy.txt #copy and rename a file
8 mv copy.txt copy1.txt #move and rename a file
9 rm copy.txt #remove
10 rm -r test #delete a directory
11 vi filename #open or make a file with vim editor
```

Vim editor

```
1 i #insert, edit mode
2 Esc #exit edit mode
3 :wq #save and exit
4 :q #exit, will mention you if you have modified the file
5 :q! #exit !without! save
```

you will use the next commands from time to time

vi somefile -> press i into edit mode -> do something -> esc exit edit mode -> :wq

Some tricks that can help you:

Press the `TAB` key to automatically complete some of your commands, such as file names. Pressing it twice will show you the available commands/file names you can input.

Press `↑` to retrieve previous commands.

Lesson 2: Basic ROOT exercises

Enter the ROOT environment:

```
$ root
```

```
% root
```

```

-----
| Welcome to ROOT 6.32.06                                     https://root.cern |
| (c) 1995–2024, The ROOT Team; conception: R. Brun, F. Rademakers |
| Built for macosxarm64 on Sep 21 2024, 18:21:53              |
| From tags/6-32-06@6-32-06                                  |
| With Apple clang version 15.0.0 (clang-1500.1.0.2.5)        |
| Try '.help'/'?', '.demo', '.license', '.credits', '.quit'/'q' |
-----

```

```
root [0] █
```

Use `root [0] .q` to exit ROOT.

ROOT can interpret commands one line at a time.

```

root [0] 1+1
(int) 2

```

You can also interpret commands while entering them, or compile and run a macro file with `root yourprogram.C`.

For example, enter the following commands in ROOT to create an empty histogram:

```

TH1F* graph=new TH1F("name","title",200,2.97,3.03);
graph->Draw();

```

An empty histogram is not very interesting, so use `Fill` to add data

```
% root
```

```

-----
| Welcome to ROOT 6.32.06                                     https://root.cern |
| (c) 1995–2024, The ROOT Team; conception: R. Brun, F. Rademakers |
| Built for macosxarm64 on Sep 21 2024, 18:21:53              |
| From tags/6-32-06@6-32-06                                  |
| With Apple clang version 15.0.0 (clang-1500.1.0.2.5)        |
| Try '.help'/'?', '.demo', '.license', '.credits', '.quit'/'q' |
-----

```

```

root [0] TH1F* graph=new TH1F("name","title",200,2.97,3.03);
root [1] graph->Draw();
Info in <TCanvas::MakeDefCanvas>: created default TCanvas with name c1
root [2] graph->Fill(2.99);
root [3] graph->Draw();
root [4] █

```

You can also create a macro file to hold a longer program.

Get start!

```
$ vi test.C
```

//push `i` to enter the insert mode//

```

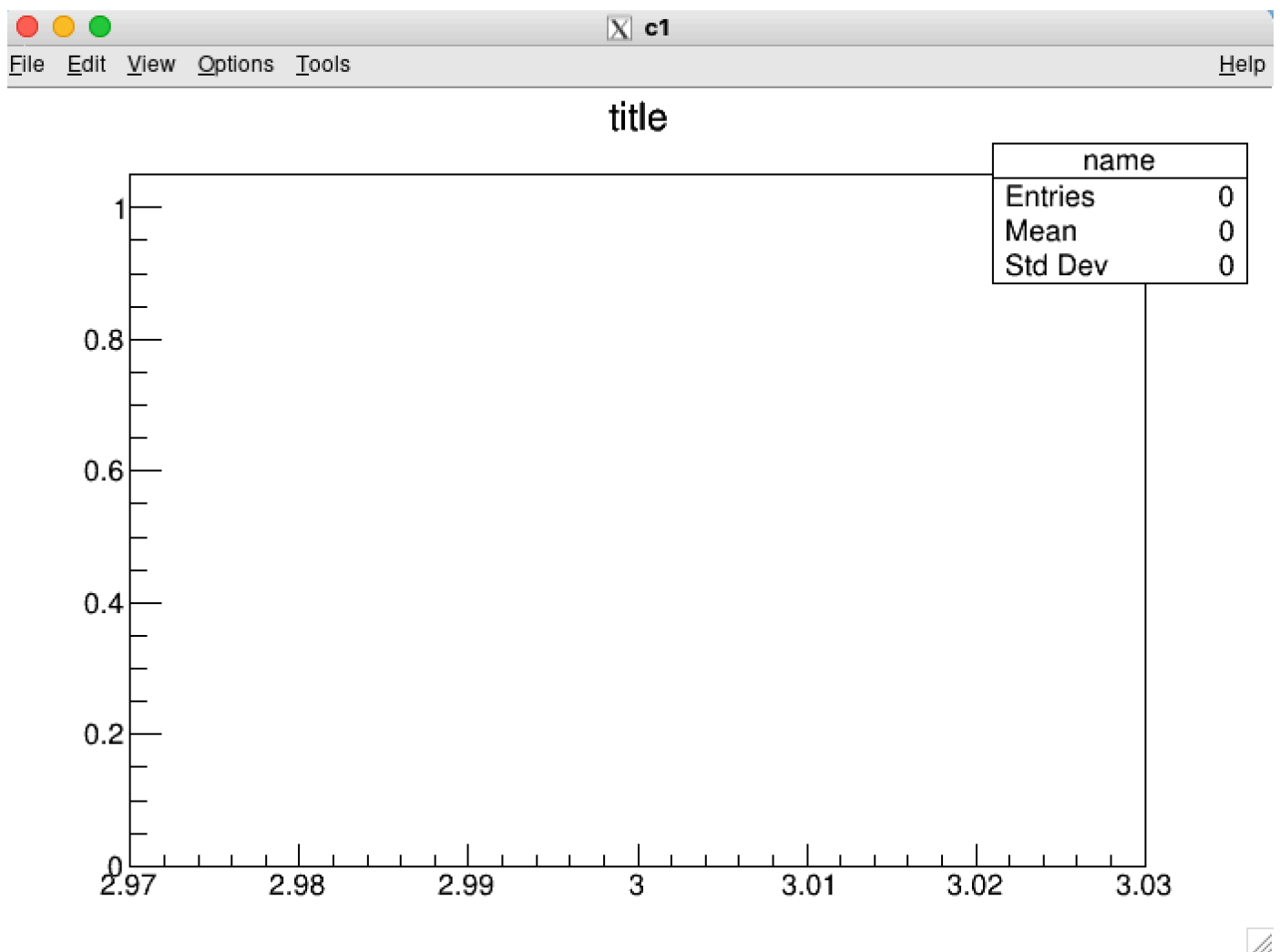
1 void test() //be sure it is similar with your file name
2
3 {
4
5     TH1F* graph=new TH1F("name","title",200,2.97,3.03);
6     //TH1F* graph name=new TH1F("name","title",bins, low, high)
7
8     graph->Draw(); //draw your graph
9
10 }

```

//push `ESC` to end the insert mode//

`:wq` //save and exit

`$ root test.c` //run your code



///successful!!!

`.q` //exit the program

Let's add something into your graph!

```

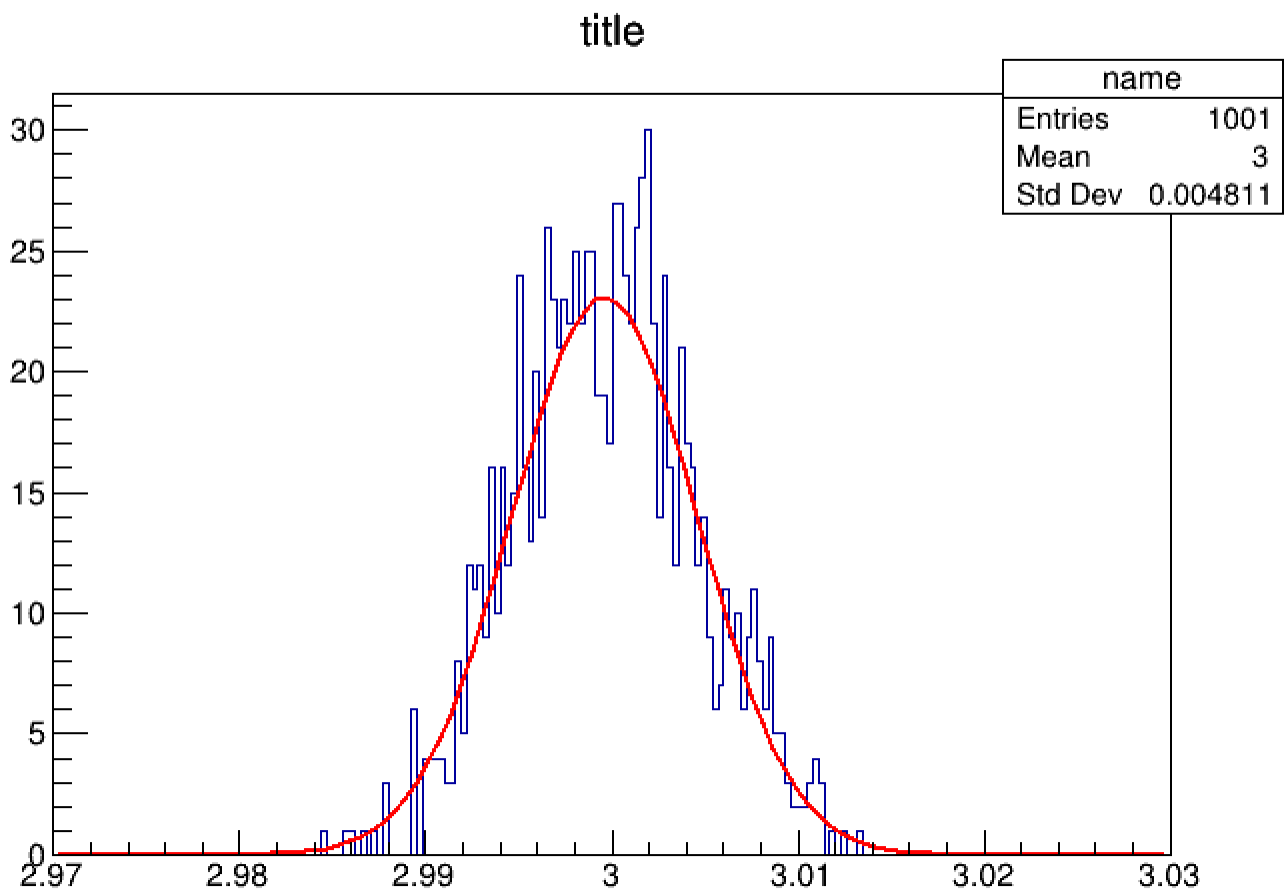
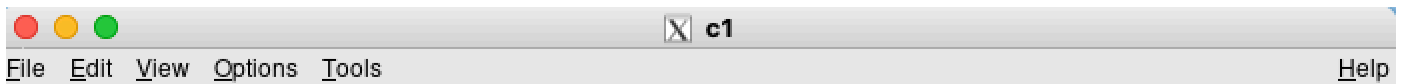
1 void test(){
2

```

```

3 //TCanvas* c1=new TCanvas("c1","fitting with Gaussian function"); //if you don't set the
  canvas's name, it will created default canvas with name c1
4
5 //c1->SetGrid(); // set grid
6
7 TH1F* graph=new TH1F("name","title",200,2.97,3.03);
8
9 graph->Fill(2.99); // you can add just one data in your graph using "Fill" command and try to
  draw your graph.It's boring, right? We need more!
10
11 TRandom n; // define a random variable n
12
13 for (int i=0;i<1000;++i){
14
15   graph->Fill(n.Gaus(3,0.005)); //Determine variable n using Gaussian distribution and fill to
  graph
16
17 }
18
19 graph->Draw();
20
21 graph->Fit("gaus"); //fit your graph with Gaussian function
22
23 }

```



Wow! now you can use root to do some simple fit works. but we usually use RooFit to do more complex job! So, ready for more programs!

These are only basic fits. Many parameters cannot be defined freely, while `RooFit` provides more professional and flexible fitting functions.

Lesson 2 Gaussian fitting using RooFit

```

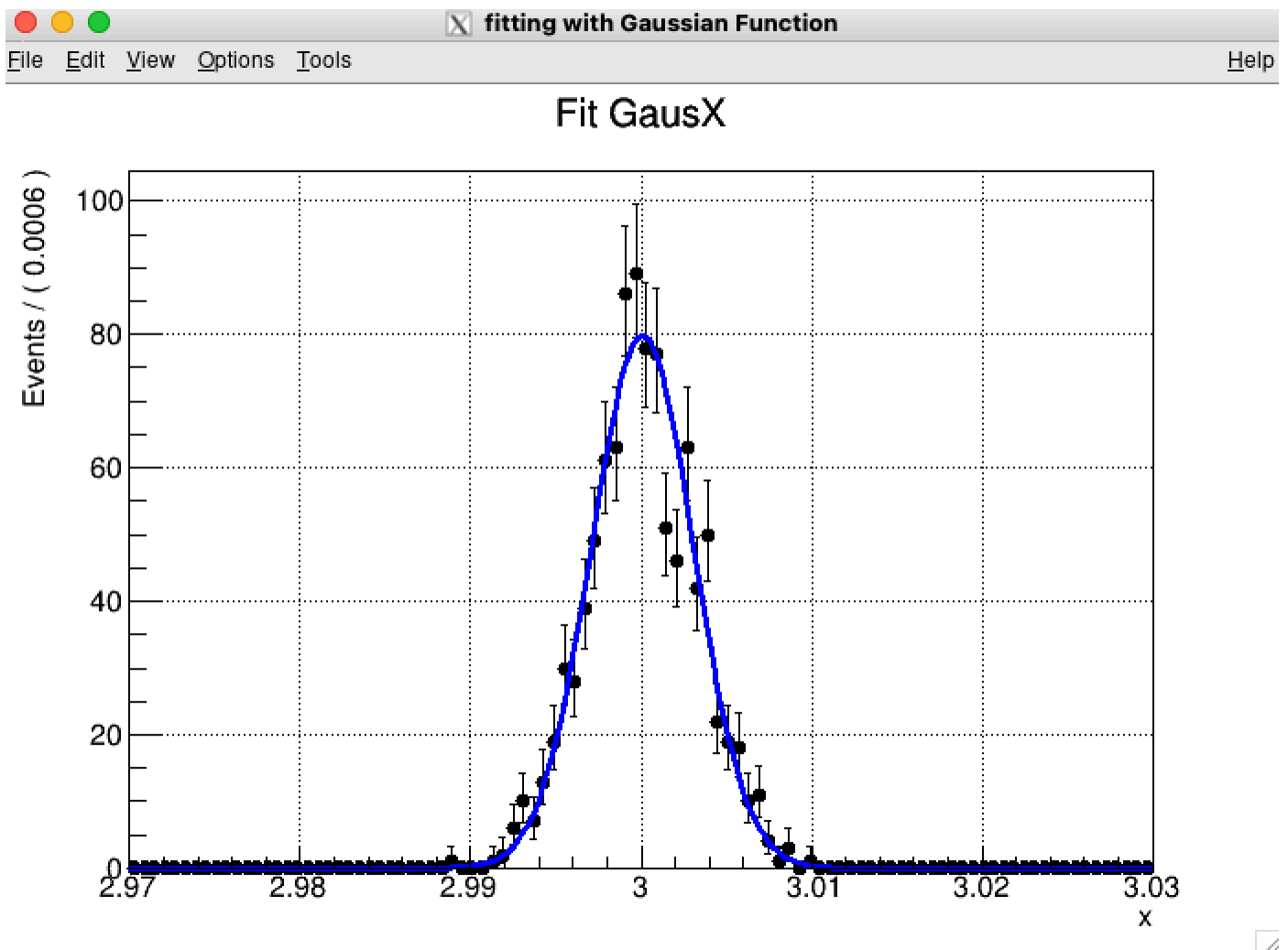
1  #include "RooAbsReal.h"
2  #include "RooRealVar.h"
3  #include "RooGaussian.h"
4  #include "RooChebychev.h"
5  #include "RooAddPdf.h"
6  //#include "RooProdPdf.h"
7  #include "RooDataSet.h"
8  //#include "RooDataHist.h"
9  //#include "RooFitResult.h"
10 #include "RooPlot.h"
11 //#include "RooArgList.h"
12 //#include "RooArgSet.h"
13 //#include "RooRandom.h"
14 //#include "RooPrintable.h"
15 using namespace RooFit;
16 void test() {
17     RooRealVar x("x", "x", 2.97, 3.03);
18     // Declares a real-valued variable 'x' with a range from 2.97 to 3.03.
19
20     RooRealVar mean("mean", "mean", 3.0, 2.8, 3.2);
21     // Declares a real-valued variable 'mean' with an initial value of 3.0 and a range from 2.8 to 3.2.
22
23     RooRealVar sigma("sigma", "sigma", 0.003, 0.002, 0.003);
24     // Declares a real-valued variable 'sigma' with an initial value of 0.003 and a range from 0.002 to 0.003.
25
26     RooAbsPdf* gaus = new RooGaussian("gaus", "gaus", x, mean, sigma);
27     // Creates a Gaussian probability density function (PDF) named 'gaus' with 'x' as the
28     // observable, 'mean' as the mean, and 'sigma' as the standard deviation.
29
30     RooRealVar n("n", "n", 0, 0, 50000);
31     // Declares a real-valued variable 'n' with an initial value of 0 and a range from 0 to 50000.
32
33     RooExtendPdf* exp = new RooExtendPdf("exp", "exp", *gaus, n);
34     // Creates an extended PDF named 'exp' that combines the Gaussian PDF 'gaus' with the variable 'n'.
35
36     RooAddPdf total("total", "total", RooArgList(*exp), RooArgList(n));
37     // Creates a composite PDF named 'total' that consists of the extended PDF 'exp' and the
38     // variable 'n'.
39
40     RooDataSet* data;
41     // Declares a pointer to a RooDataSet object named 'data'.
42
43     data = gaus->generate(RooArgSet(x), 1000);
44     // Generates a dataset 'data' with 1000 events based on the Gaussian PDF 'gaus' and the
45     // observable 'x'.
46
47     RooFitResult* result = total.fitTo(*data, Save());

```

```

45 // Fits the composite PDF 'total' to the dataset 'data' and saves the fit result in 'result'.
46
47 TCanvas* c1 = new TCanvas("c1", "fitting with Gaussian function");
48 // Creates a new canvas named 'c1' with the title "fitting with Gaussian function" for
plotting.
49
50 RooPlot* xframe = x.frame(RooFit::Title("Fit GausX"));
51 // Creates a frame for the observable 'x' with the title "Fit GausX".
52
53 data->plotOn(xframe);
54 // Plots the dataset 'data' on the frame 'xframe'.
55
56 total.plotOn(xframe);
57 // Plots the composite PDF 'total' on the frame 'xframe'.
58
59 xframe->Draw();
60 // Draws the frame 'xframe' on the canvas.
61 }

```



One function can look a little lonely. We can use `RooAddPdf` to combine two functions.

```

RooRealVar fsig("fsig","fsig",0.2,0,1.);
//model(x) = fsiggaus(x) + (1-fsig)chev(x)
RooAddPdf model("fsig","fsig",RooArgList(gaus,chev),fsig);

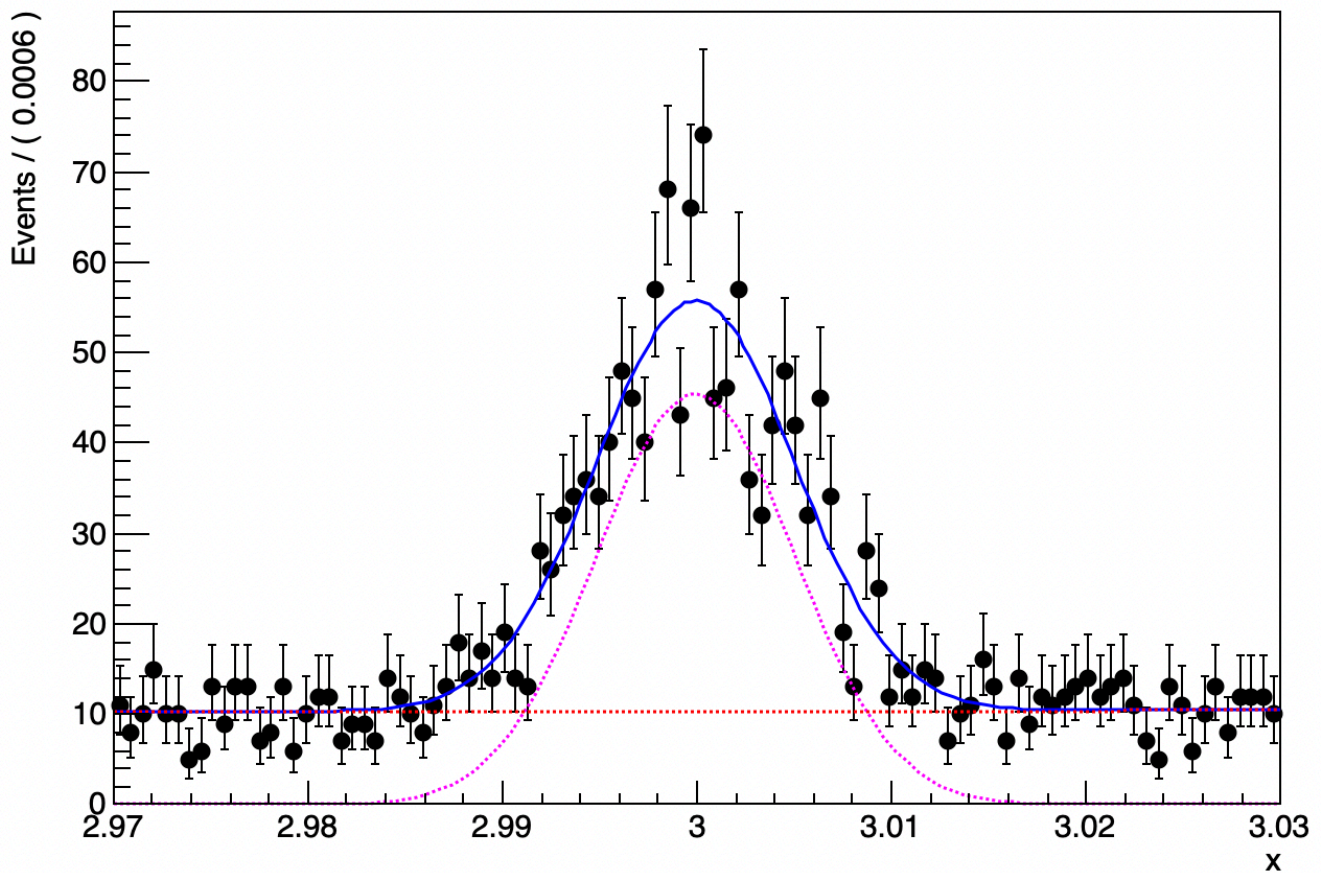
```

```

1 void test(){
2   using namespace RooFit;
3   //build gaussian and shev pdf
4   RooRealVar x("x","x",2.97,3.03);
5   RooRealVar mean("mean","mean",3.0,2.8,3.2);
6   RooRealVar sigma("sigma","sigma",0.005,0.003,0.007);
7   RooRealVar a0("a0","a0",0.005,0.004,0.006);
8   RooAbsPdf* gaus=new RooGaussian("gaus","gaus",x,mean,sigma);
9   RooAbsPdf* shev=new RooChebychev("shev","shev",x,a0);
10  RooRealVar n1("n1","n1",0,0,50000);
11  RooRealVar n2("n2","n2",0,0,50000);
12  RooExtendPdf* signal1=new RooExtendPdf("sig1","sig1",*gaus,n1);
13  RooExtendPdf* signal2=new RooExtendPdf("sig2","sig2",*shev,n2);
14  RooAddPdf
    totalPdf("total","total",RooArgList(*signal1,*signal2),RooArgList(n1,n2)); //totalPdf(x)=signal1(x)
    +signal2(x)
15
16  //generate data point
17  RooDataSet*data1;
18  RooDataSet*data2;
19  data1=gaus->generate(RooArgSet(x),1000);
20  data2=shev->generate(RooArgSet(x),1000);
21  data1->append(*data2);
22
23  //fit
24  RooFitResult*result= totalPdf.fitTo(*data1,Save());
25  RooPlot*xframe=x.frame();
26  data1->plotOn(xframe);
27  totalPdf.plotOn(xframe);
28  totalPdf.plotOn(xframe, Components(*signal1),LineStyle(kDashed),LineColor(6));
29  totalPdf.plotOn(xframe, Components("sig2"),LineStyle(kDashed),LineColor(2)); //two different ways
    set the Components name
30  xframe->Draw();
31 }

```

A RooPlot of "x"



The example above is a one-dimensional fit. In many analyses we study two different components, which requires a two-dimensional fit. A two-dimensional fit does not simply add two PDFs; it uses products, which can be understood as the probability that two components occur together. For an x event with probabilities $p(x)$ and $q(x)$, and a y event with probabilities $p(y)$ and $q(y)$, there are four possible combinations. Their probabilities include $p(x)*p(y)$, the probability that x and y occur together. We therefore build four probability functions for the different combinations.

Note: The default random seed is `0`. A seed is usually based on the current time so that each run produces different data, but a fixed seed is convenient for debugging. With seed `0`, a program like this produces the same data every time. You can set the seed with `RooRandom::randomGenerator()->SetSeed(seed)` or with `gRandom->SetSeed(seed)`. The first controls RooFit's random generator; the second controls ROOT's global random generator and can also cover uses outside RooFit. Use `gRandom->GetSeed()` to inspect the current seed.

```
1 TDateTime* starttime=new TDateTime();
2 Int_t today=starttime->GetDate();
3 Int_t clock=starttime->GetTime();
4 Int_t seed=today+clock;
5 RooRandom::randomGenerator()->SetSeed(seed);
```

Lesson 3 2D fitting

The following section shows two two-dimensional fits.

```
1 #include <TH2.h>
2 #include <TStyle.h>
3 #include <TCanvas.h>
```

```

4  #include <iostream>
5  #include <fstream>
6  #include "TF1.h"
7  #include "TText.h"
8  #include "TLorentzVector.h"
9  #include "TVector3.h"
10 #include "TLorentzRotation.h"
11 #include "RooNumIntConfig.h" //no matter what, copy directly!
12
13 using namespace RooFit;
14 void test(){
15 //build gaussian pdf in two dimension
16 RooRealVar x("x", "x", 2.7, 4);
17
18 RooRealVar meanx("meanx", "meanx", 3.0967, 2.9, 3.2);
19 RooRealVar sigmax("sigmax", "sigmax", 0.03, 0.01, 0.06);
20 RooAbsPdf* gausx=new RooGaussian("gausx", "gausx", x, meanx, sigmax);
21
22 RooRealVar ax0("ax0", "ax0", 0.005, 0.004, 0.006);
23 RooRealVar ax1("ax1", "ax1", 0.01, 0.01, 0.02);
24 RooAbsPdf* shevx=new RooChebychev("shevx", "shevx", x, RooArgSet(ax0, ax1));
25
26 RooRealVar y("y", "y", 2.7, 4);
27
28 RooRealVar meany("many", "many", 3.0967, 2.9, 3.2);
29 RooRealVar sigmay("sigmay", "sigmay", 0.03, 0.01, 0.06);
30 RooAbsPdf* gausy=new RooGaussian("gausy", "gausy", y, meany, sigmay);
31
32 RooRealVar ay0("ay0", "ay0", 0.005, 0.004, 0.006);
33 RooRealVar ay1("ay1", "ay1", 0.01, 0.01, 0.02);
34 RooAbsPdf* shevy=new RooChebychev("shevy", "shevy", y, RooArgSet(ay0, ay1));
35
36 //constructor with 2 pdf
37 RooProdPdf* sigxsigy=new RooProdPdf("sigxsigy", "sigxsigy", *gausx, *gausy);
38 RooProdPdf* bkgxsigy=new RooProdPdf("bkgxsigy", "bkgxsigy", *shevx, *gausy);
39 RooProdPdf* sigxbkgy=new RooProdPdf("sigxbkgy", "sigxbkgy", *gausx, *shevy);
40 RooProdPdf* bkgxbkgy=new RooProdPdf("bkgxbkgy", "bkgxbkgy", *shevx, *shevy);
41
42 RooRealVar n_sigxsigy("n_sigxsigy", "n_sigxsigy", 0, 0, 50000);
43 RooRealVar n_bkgxsigy("n_bkgxsigy", "n_bkgxsigy", 0, 0, 50000);
44 RooRealVar n_sigxbkgy("n_sigxbkgy", "n_sigxbkgy", 0, 0, 50000);
45 RooRealVar n_bkgxbkgy("n_bkgxbkgy", "n_bkgxbkgy", 0, 0, 50000);
46
47
48 RooExtendPdf*signal1=new RooExtendPdf("signal1", "signal1", *sigxsigy, n_sigxsigy);
49 RooExtendPdf*signal2=new RooExtendPdf("signal2", "signal2", *sigxbkgy, n_sigxbkgy);
50 RooExtendPdf*signal3=new RooExtendPdf("signal3", "signal3", *bkgxsigy, n_bkgxsigy);
51 RooExtendPdf*signal4=new RooExtendPdf("signal4", "signal4", *bkgxbkgy, n_bkgxbkgy);
52
53 RooAddPdf
54 totalPdf("total", "total", RooArgList(*signal1, *signal2, *signal3, *signal4), RooArgList(n_sigxsigy, n
55 _sigxbkgy, n_bkgxsigy, n_bkgxbkgy));
56
57 //generate data point
58
59 RooDataSet*data1;
60 RooDataSet*data2;
61 RooDataSet*data3;

```

```

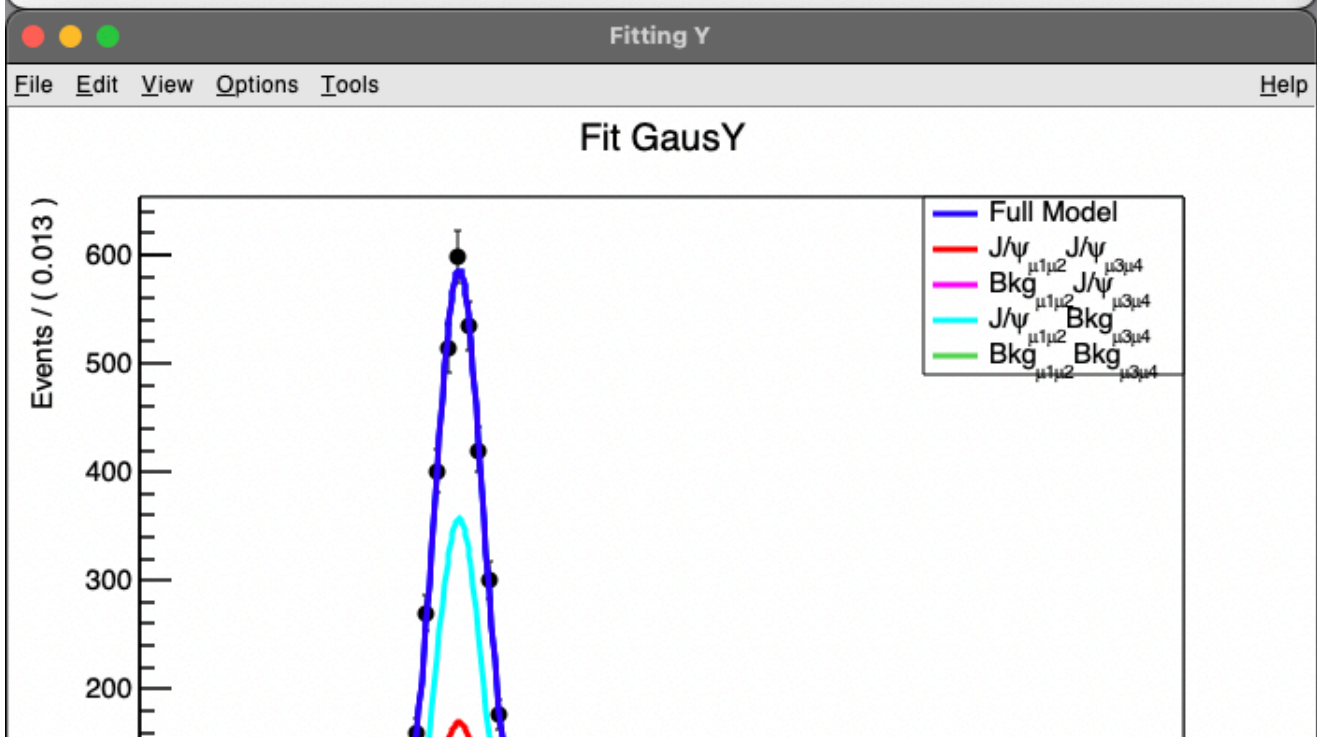
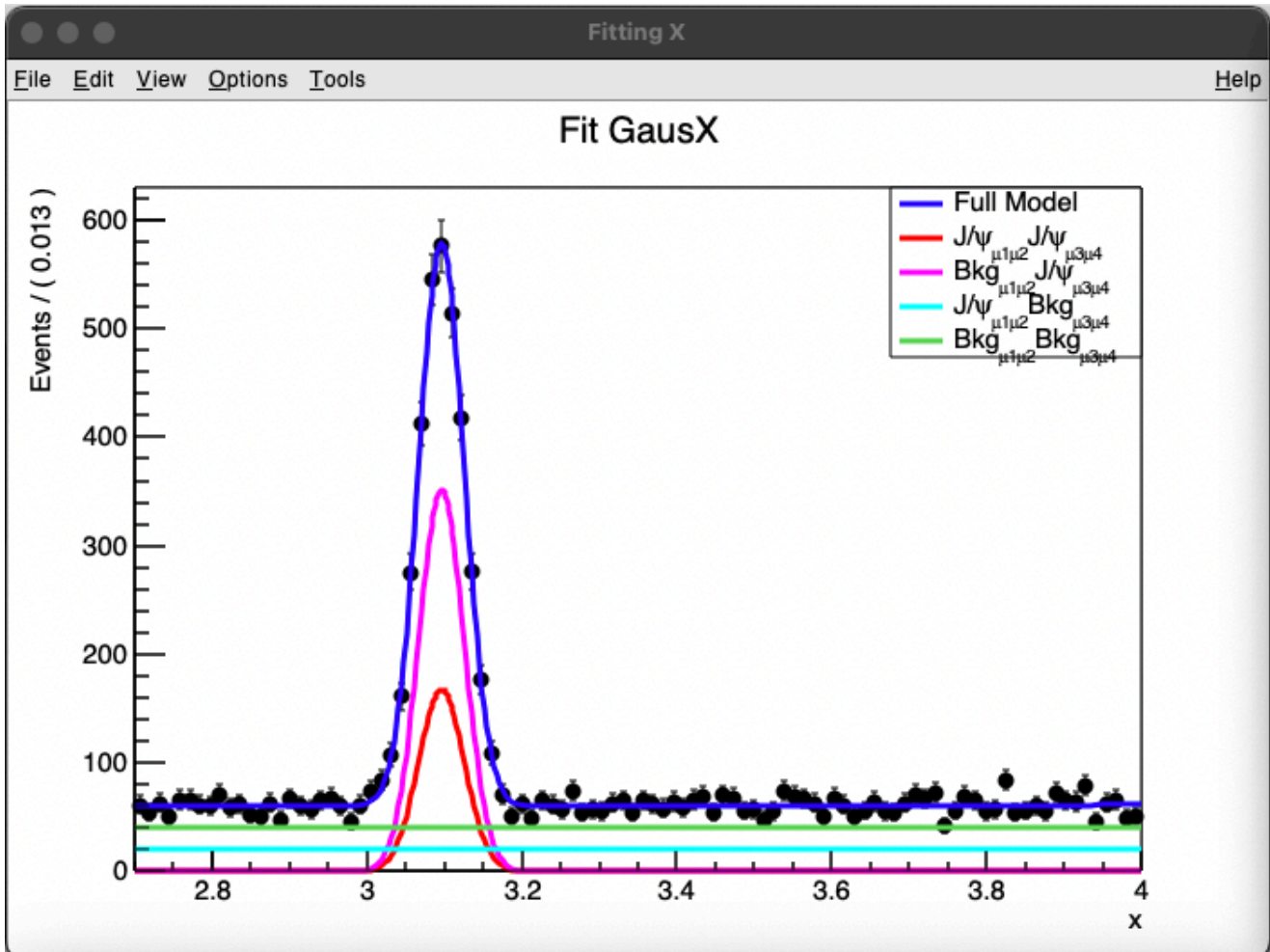
60 RooDataSet*data4;
61 data1=sigxsigy->generate(RooArgSet(x,y),1000);
62 data2=sigxbkgy->generate(RooArgSet(x,y),2000);
63 data3=bkgxsigy->generate(RooArgSet(x,y),2000);
64 data4=bkgxbkgy->generate(RooArgSet(x,y),4000);
65
66 data1->append(*data2);
67 data1->append(*data3);
68 data1->append(*data4);
69
70 //fit
71 RooFitResult*result= totalPdf.fitTo(*data1,Save());
72 //x dimension
73
74 //create two canvases, because you have to know two dimensions fitting.
75 TCanvas*c1 =new TCanvas("c1","Fitting X");
76 c1->cd();
77 RooPlot*xframe=x.frame(RooFit::Title("Fit GausX"));
78 data1->plotOn(xframe);
79 totalPdf.plotOn(xframe,Name("fullModel"));
80 totalPdf.plotOn(xframe,Components(*signal1),LineColor(2),LineStyle(1),Name("JpsiJpsi"));
81 totalPdf.plotOn(xframe,Components(*signal2),LineColor(6),LineStyle(1),Name("BkgJpsi"));
82 totalPdf.plotOn(xframe,Components(*signal3),LineColor(7),LineStyle(1),Name("JpsiBkg"));
83 totalPdf.plotOn(xframe,Components(*signal4),LineColor(8),LineStyle(1),Name("BkgBkg"));
84
85 TLegend leg(0.7, 0.7, 0.9, 0.9);
86 leg.AddEntry(xframe->findObject("fullModel"), "Full Model", "L");
87 leg.AddEntry(xframe->findObject("JpsiJpsi"), "J/#psi_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
88 leg.AddEntry(xframe->findObject("BkgJpsi"), "Bkg_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
89 leg.AddEntry(xframe->findObject("JpsiBkg"), "J/#psi_{#mu1#mu2}Bkg_{#mu3#mu4}", "L");
90 leg.AddEntry(xframe->findObject("BkgBkg"), "Bkg_{#mu1#mu2}Bkg_{#mu3#mu4}", "L");
91
92 xframe->Draw();
93 leg.DrawClone();
94
95 //y dimension
96 TCanvas*c2 =new TCanvas("c2","Fitting Y");
97 c2->cd();
98 RooPlot*yframe=y.frame(RooFit::Title("Fit GausY"));
99 data1->plotOn(yframe);
100 totalPdf.plotOn(yframe);
101 totalPdf.plotOn(yframe,Name("fullModel"));
102 totalPdf.plotOn(yframe,Components(*signal1),LineColor(2),LineStyle(1),Name("JpsiJpsi"));
103 totalPdf.plotOn(yframe,Components(*signal2),LineColor(6),LineStyle(1),Name("BkgJpsi"));
104 totalPdf.plotOn(yframe,Components(*signal3),LineColor(7),LineStyle(1),Name("JpsiBkg"));
105 totalPdf.plotOn(yframe,Components(*signal4),LineColor(8),LineStyle(1),Name("BkgBkg"));
106
107 TLegend leg2(0.7, 0.7, 0.9, 0.9);
108 leg2.AddEntry(yframe->findObject("fullModel"), "Full Model", "L");
109 leg2.AddEntry(yframe->findObject("JpsiJpsi"), "J/#psi_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
110 leg2.AddEntry(yframe->findObject("BkgJpsi"), "Bkg_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
111 leg2.AddEntry(yframe->findObject("JpsiBkg"), "J/#psi_{#mu1#mu2}Bkg_{#mu3#mu4}", "L");
112 leg2.AddEntry(yframe->findObject("BkgBkg"), "Bkg_{#mu1#mu2}Bkg_{#mu3#mu4}", "L");
113
114 yframe->Draw();
115 leg2.DrawClone();
116
117 TCanvas*c3 =new TCanvas("c3","2D plot");

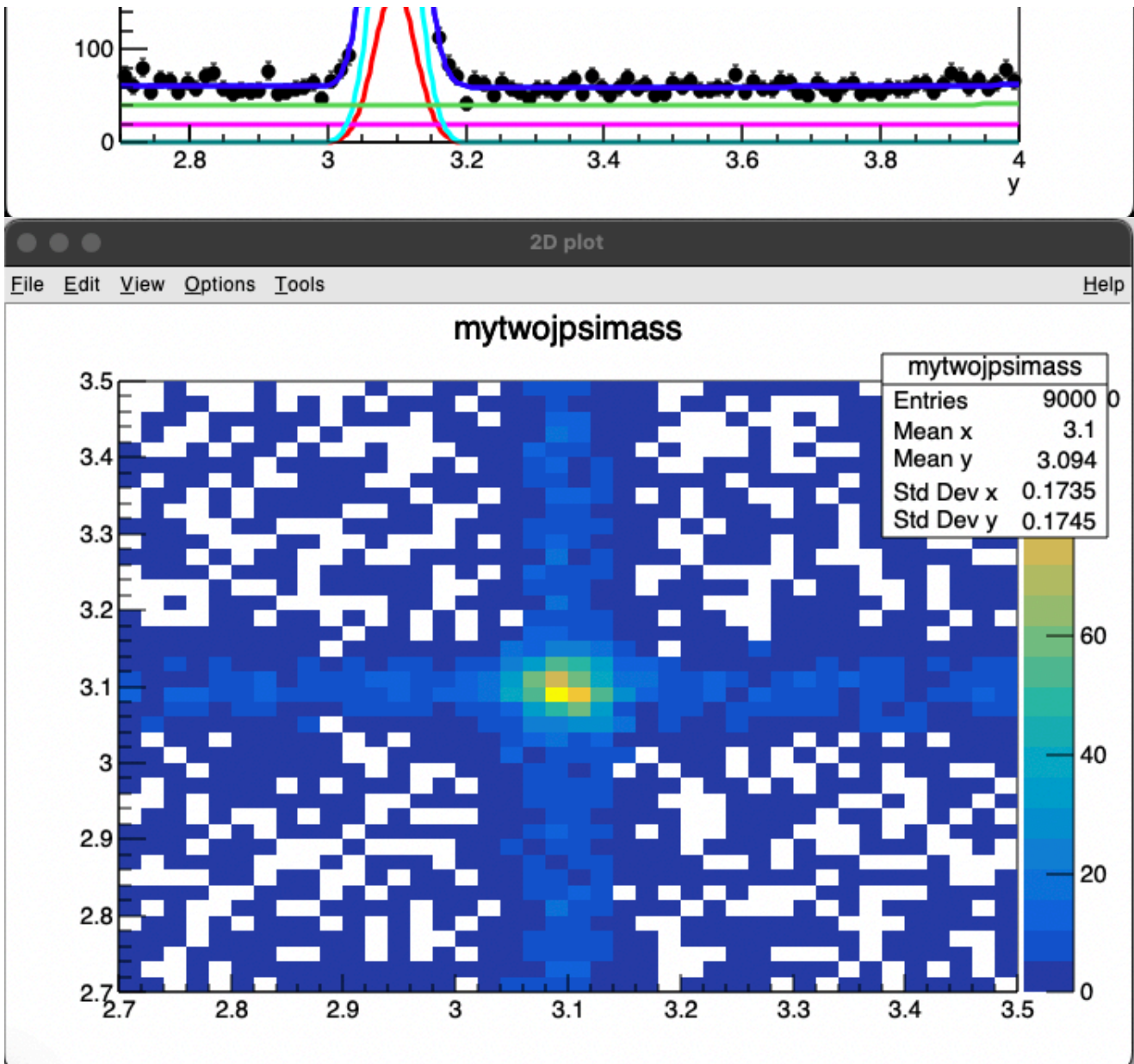
```

```

118 c3->cd();
119 TH2F* mytwojpsimass = new TH2F("mytwojpsimass","mytwojpsimass",40,2.7,3.5,40,2.7,3.5);
120 data1->fillHistogram(mytwojpsimass,RooArgList(x,y));
121 mytwojpsimass->Draw("colz");
122
123 }

```





```

1  #include <TH2.h>
2  #include <TStyle.h>
3  #include <TCanvas.h>
4  #include <iostream>
5  #include <fstream>
6  #include "TF1.h"
7  #include "TText.h"
8  #include "TLorentzVector.h"
9  #include "TVector3.h"
10 #include "TLorentzRotation.h"
11 #include "RooNumIntConfig.h" //no matter what, copy directly!
12 using namespace RooFit;
13 void test(){
14 //build gaussian pdf in two dimension
15 RooRealVar x("x", "x", 2.7, 4);
16 RooRealVar meanx1("meanx1", "meanx1", 3.0967, 2.9, 3.2);
17 RooRealVar sigmax1("sigmax1", "sigmax1", 0.03, 0.01, 0.06);
18 RooAbsPdf* gausx1=new RooGaussian("gausx1", "gausx1", x, meanx1, sigmax1);
19
20 RooRealVar meanx2("meanx2", "meanx2", 3.686, 3.5, 3.8);

```

```

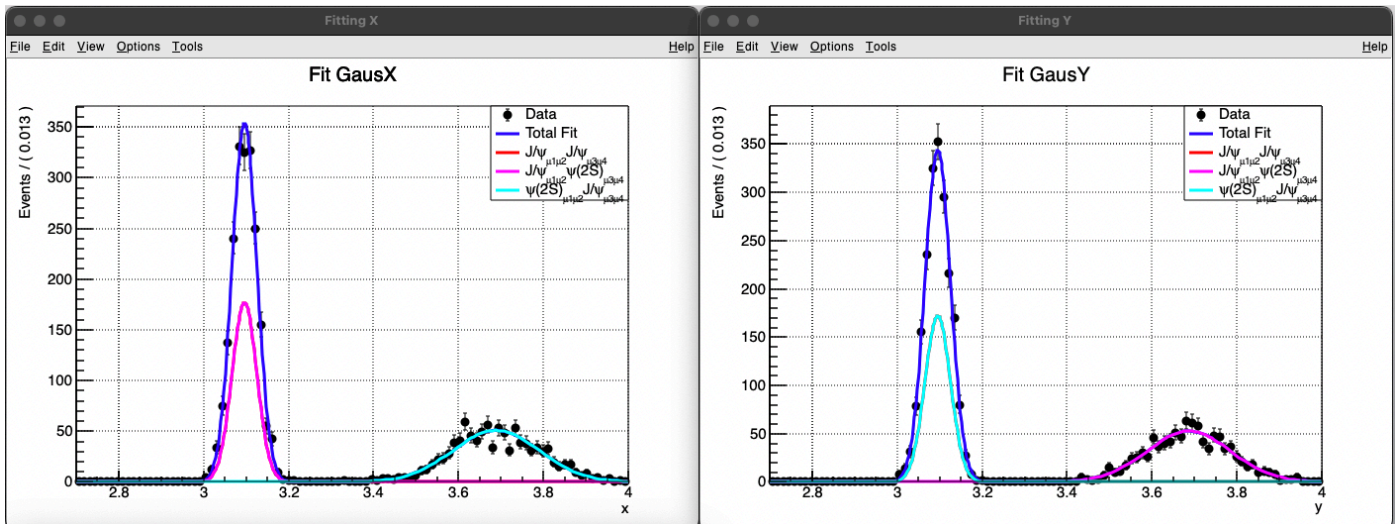
21 RooRealVar sigmax2("sigmax2","sigmax2",0.1,0.01,0.2);
22 RooAbsPdf* gausx2=new RooGaussian("gausx2","gausx2",x,meanx2,sigmax2);
23
24 RooRealVar y("y","y",2.7,4);
25
26 RooRealVar meany1("meany1","meany1",3.0967,2.9,3.2);
27 RooRealVar sigmay1("sigmay1","sigmay1",0.03,0.01,0.06);
28 RooAbsPdf* gausy1=new RooGaussian("gausy1","gausy1",y,meany1,sigmay1);
29
30 RooRealVar meany2("meany2","meany2",3.686,3.5,3.8);
31 RooRealVar sigmay2("sigmay2","sigmay2",0.1,0.01,0.2);
32 RooAbsPdf* gausy2=new RooGaussian("gausy2","gausy2",y,meany2,sigmay2);
33
34 //constructor with 2 pdf
35 RooProdPdf* gausxly1=new RooProdPdf("gausxly1","gausxly1",*gausx1,*gausy1);
36 RooProdPdf* gausxly2=new RooProdPdf("gausxly2","gausxly2",*gausx1,*gausy2);
37 RooProdPdf* gausx2y1=new RooProdPdf("gausx2y1","gausx2y1",*gausx2,*gausy1);
38
39 RooRealVar n_xly1("n1","n1",0,0,50000);
40 RooRealVar n_xly2("n2","n2",0,0,50000);
41 RooRealVar n_x2y1("n3","n3",0,0,50000);
42
43 RooAddPdf
totalPdf("total","total",RooArgList(*gausxly1,*gausxly2,*gausx2y1),RooArgList(n_xly1,n_xly2,n_x2
y1));
44
45 //generate data point
46
47 RooDataSet*data1;
48 RooDataSet*data2;
49 RooDataSet*data3;
50 data1=gausxly1->generate(RooArgSet(x,y),1000);
51 data2=gausxly2->generate(RooArgSet(x,y),1000);
52 data3=gausx2y1->generate(RooArgSet(x,y),1000);
53
54 data1->append(*data2);
55 data1->append(*data3);
56
57 //fit
58 RooFitResult*result= totalPdf.fitTo(*data1,Save());
59 //x dimension
60 //create two canvases, because you have to know two dimensions fitting.
61 TCanvas*c1 =new TCanvas("c1","Fitting X");
62 c1->cd();
63 RooPlot*xframe=x.frame(RooFit::Title("Fit GausX"));
64 data1->plotOn(xframe,Name("data1"));
65 totalPdf.plotOn(xframe,Name("fullModel"));
66 totalPdf.plotOn(xframe,Components(*gausxly1),LineColor(2),LineStyle(1),Name("JpsiJpsi"));
67 totalPdf.plotOn(xframe,Components(*gausxly2),LineColor(6),LineStyle(1),Name("JpsiPsi2S"));
68 totalPdf.plotOn(xframe,Components(*gausx2y1),LineColor(7),LineStyle(1),Name("Psi2SJpsi"));
69
70 TLegend* leg= new TLegend(0.7, 0.7, 0.9, 0.9);
71 leg->AddEntry(xframe->findObject("data1"), "Data", "pe");
72 leg->AddEntry(xframe->findObject("fullModel"), "Total Fit", "L");
73 leg->AddEntry(xframe->findObject("JpsiJpsi"), "J/#psi_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
74 leg->AddEntry(xframe->findObject("JpsiPsi2S"), "J/#psi_{#mu1#mu2}#psi(2S)_{#mu3#mu4}", "L");
75 leg->AddEntry(xframe->findObject("Psi2SJpsi"), "#psi(2S)_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
76

```

```

77 xframe->Draw();
78 leg->Draw();
79
80
81
82 //y dimension
83 TCanvas*c2 =new TCanvas("c2","Fitting Y");
84 c2->cd();
85 RooPlot*yframe=y.frame(RooFit::Title("Fit GausY"));
86 data1->plotOn(yframe,Name("data1"));
87 totalPdf.plotOn(yframe,Name("fullModel"));
88 totalPdf.plotOn(yframe,Components(*gausx1y1),LineColor(2),LineStyle(1),Name("JpsiJpsi"));
89 totalPdf.plotOn(yframe,Components(*gausx1y2),LineColor(6),LineStyle(1),Name("JpsiPsi2S"));
90 totalPdf.plotOn(yframe,Components(*gausx2y1),LineColor(7),LineStyle(1),Name("Psi2SJpsi"));
91
92
93 TLegend leg2(0.7, 0.7, 0.9, 0.9);
94 leg2.AddEntry(yframe->findObject("data1"), "Data", "pe");
95 leg2.AddEntry(yframe->findObject("fullModel"), "Total Fit", "L");
96 leg2.AddEntry(yframe->findObject("JpsiJpsi"), "J/#psi_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
97 leg2.AddEntry(yframe->findObject("JpsiPsi2S"), "J/#psi_{#mu1#mu2}#psi(2S)_{#mu3#mu4}", "L");
98 leg2.AddEntry(yframe->findObject("Psi2SJpsi"), "#psi(2S)_{#mu1#mu2}J/#psi_{#mu3#mu4}", "L");
99
100 yframe->Draw();
101 leg2.DrawClone();
102
103 TCanvas*c3 =new TCanvas("c3","2D plot");
104 c3->cd();
105 TH2F* mytwojpsimass = new TH2F("mytwojpsimass", "mytwojpsimass",40,2.7,4,40,2.7,4);
106 data1->fillHistogram(mytwojpsimass,RooArgList(x,y));
107 mytwojpsimass->Draw("colz");

```



Lesson 3+ Pull Plot

To evaluate how well the fit describes the data, inspect the pull distribution. It straightens the fitted curve and shows the distance of each data point from the curve. The pull plot for `xframe` is shown below.

```

1 RooPlot *xfpull=x.frame(RooFit::Title("      ")); //create the RooPlot object xfpull for the pull
  distribution along x
2 RooHist *pullx=xframe->pullHist("data","fullModel"); //get the data/model pull from xframe; the
  names must match the objects drawn on the frame

```

```

3  xfpull->addPlotable(pullx,"p");
4  xfpull->GetXaxis()->SetTitle("J/#psi_{#mu1#mu2}");
5  xfpull->GetXaxis()->SetTitleSize(0.15);
6  xfpull->GetXaxis()->SetLabelSize(0.12);
7  xfpull->GetYaxis()->SetTitle("Pull");
8  xfpull->GetYaxis()->SetTitleSize(0.15);
9  xfpull->GetYaxis()->SetLabelSize(0.1);
10 xfpull->GetYaxis()->SetTitleOffset(0.2);
11
12 //在画布 cpull 上创建两个子画布 pad11 和 pad12, 分别用于绘制拟合结果和 pull 分布图。
13 //在 pad11 上绘制 y 维度的拟合结果 xframe 和图例 leg, TPad的位置和TLegend的类似, 都是相对于TCanvas的位置关系
14 TCanvas cpull("cpull","cpull",800,600);
15 cpull.cd();
16 TPad pad11("pad11","pad11",0,0.3,1,1.0);
17 pad11.SetTopMargin(0.08);
18 pad11.SetBottomMargin(0.005);
19 pad11.Draw();
20 pad11.cd();
21 xframe->Draw();
22 leg.Draw();
23 //在 pad12 上绘制 y 维度的 pull 分布图 yfpull
24 cpull.cd();
25 TPad pad12("pad12","pad12",0,0.0,1,0.3);
26 pad12.SetTopMargin(0.005);
27 pad12.SetBottomMargin(0.4);
28 pad12.SetGridx();
29 pad12.SetGridy(2);
30 pad12.Draw();
31 pad12.cd();
32 xfpull->Draw();
33
34 cpull.Update();
35 cpull.SaveAs("cpull.pdf");

```

